High Performance Computing (HPC)

Assignment 06 – Parallel Interpolation

Name: OM PATEL
Roll Number: 202301163

Name: CHAITANYA VATS
Roll Number: 202301419

Dhirubhai Ambani University

Instructor: Dr. Bhaskar Chaudhury

Teaching Assistants: Libin Varghese, Ayushi Sharma, Kaushik Prajapati

April 2026

Contents

1 Problem Statement **2**
1.1 Problem Description . . . 2
1.2 Step-by-Step Procedure . . . 2
1.3 Input Configurations . . . 3

2 Theoretical Background **3**
2.1 Bilinear Interpolation (Scatter Approach) . . . 3
2.2 OpenMP Parallelism . . . 3
2.3 Race Conditions in Parallel Scatter . . . 3
2.4 Amdahl's Law and Parallel Efficiency . . . 4

3 Implementation Details **4**
3.1 Parallelization Strategy: Thread-Local Grid Privatization . . . 4
3.2 Key Optimizations . . . 5
3.3 Source Code: Parallel Interpolation . . . 5

4 Results – Tabular Analysis **6**
4.1 Lab PC Results (Windows) . . . 6
4.2 HPC Cluster Results . . . 6
4.3 Speedup on HPC Cluster . . . 7
4.4 Parallel Efficiency on HPC Cluster . . . 7
4.5 Maximum Speedup Achieved . . . 7

5 Performance Visualisations **8**
5.1 HPC Cluster Results . . . 8
5.2 Lab PC Results (Windows) . . . 9
5.3 Per-Configuration Analysis (HPC Cluster) . . . 11

6 Analysis and Questions **11**
6.1 Q1: Pseudocode and Illustrative Diagram . . . 11
6.1.1 Pseudocode . . . 11
6.1.2 Illustrative Diagram . . . 12
6.2 Q2: Parallel Approach and Race Condition Handling . . . 12
6.3 Q3: Plots . . . 13
6.4 Q4: Parallel Efficiency and Scalability . . . 13
6.5 Q5: Maximum Speedup . . . 13
6.6 Q6: Explanation of Speedup and Observations . . . 14
6.7 Q7: Optimization with Unlimited HPC Resources . . . 14
6.8 Q8: Load Imbalance and Performance Bottlenecks . . . 15
6.9 Q9: Alternative Approach . . . 15
6.10 Q10: Additional Data Structures and OpenMP Approaches . . . 15

7 Conclusion **16**

1 Problem Statement

This assignment focuses on **parallelizing** a scatter point-to-mesh bilinear interpolation algorithm using **OpenMP**. MPI is not permitted for this assignment. The goal is to transform a correct serial baseline into a high-performance parallel implementation.

1.1 Problem Description

Given a set of N scattered points in a 2D domain of size 1×1:

$$S = \{(x_i, y_i, f_i) \mid i = 1, 2, \ldots, N\}$$

where (x_i, y_i) are spatial coordinates and $f_i = 1$ for all points. The task is to **interpolate** these values onto a structured mesh grid of size $M_x \times M_y$.

1.2 Step-by-Step Procedure

1. **Read input data**: Load N scattered points (x_i, y_i) from a binary file `input.bin`. The domain is a unit square $[0, 1] \times [0, 1]$.
2. **Initialize structured mesh**: Define the grid G with $(N_x + 1) \times (N_y + 1)$ grid points and set all function values $F(X_i, Y_j) = 0$.
3. **For each scattered point** (x_i, y_i):
   (a) Identify the cell containing the point by computing:
   $$i = \left\lfloor \frac{x_i}{\Delta x} \right\rfloor, \quad j = \left\lfloor \frac{y_i}{\Delta y} \right\rfloor$$
   where $\Delta x = \frac{1}{N_x}$, $\Delta y = \frac{1}{N_y}$.
   (b) Compute the local offsets from the cell's lower-left corner:
   $$l_x = x_i - i \cdot \Delta x, \quad l_y = y_i - j \cdot \Delta y$$
   (c) Compute the four bilinear weights:
   $$w_{i,j} = (\Delta x - l_x)(\Delta y - l_y) \tag{1}$$
   $$w_{i+1,j} = l_x \cdot (\Delta y - l_y) \tag{2}$$
   $$w_{i,j+1} = (\Delta x - l_x) \cdot l_y \tag{3}$$
   $$w_{i+1,j+1} = l_x \cdot l_y \tag{4}$$
   (d) Accumulate the weighted value ($f_i = 1$) onto the four surrounding grid points:
   $$F(X_i, Y_j) \mathrel{+}= w_{i,j} \tag{5}$$
   $$F(X_{i+1}, Y_j) \mathrel{+}= w_{i+1,j} \tag{6}$$
   $$F(X_i, Y_{j+1}) \mathrel{+}= w_{i,j+1} \tag{7}$$
   $$F(X_{i+1}, Y_{j+1}) \mathrel{+}= w_{i+1,j+1} \tag{8}$$
4. **Repeat** step 3 for 10 iterations, each time loading a new set of scattered points from the binary file.
5. **Output**: Write the final interpolated grid to `Mesh.out`.

1.3 Input Configurations

Five configurations are tested as specified in the assignment:

Table 1: Input configurations for benchmarking

Config	N_x	N_y	**Points**	**Grid Size**	**Maxiter**
(a)	250	100	900,000	251×101	10
(b)	250	100	5,000,000	251×101	10
(c)	500	200	3,600,000	501×201	10
(d)	500	200	20,000,000	501×201	10
(e)	1000	400	14,000,000	1001×401	10

2 Theoretical Background

2.1 Bilinear Interpolation (Scatter Approach)

Bilinear interpolation in the scatter formulation distributes values from scattered points *onto* a regular grid. Each point contributes to its four nearest grid points, with weights determined by the point's relative position within its enclosing grid cell.

For a point (x_i, y_i) located in a grid cell with corners at (X_i, Y_j), (X_{i+1}, Y_j), (X_i, Y_{j+1}), and (X_{i+1}, Y_{j+1}):

$$w_{i,j} = (\Delta x - l_x)(\Delta y - l_y)$$

The weight for each corner is proportional to the area of the rectangle **opposite** to that corner. This ensures that the total weight sums to $\Delta x \cdot \Delta y$, preserving conservation.

2.2 OpenMP Parallelism

OpenMP (Open Multi-Processing) is a shared-memory parallel programming API. Key constructs used in our implementation:

- `#pragma omp parallel`: Creates a team of threads.
- `#pragma omp for schedule(static)`: Distributes loop iterations equally among threads. Ideal when each iteration has uniform cost.
- `#pragma omp single`: Ensures only one thread executes a block (used for setup).
- `omp_get_wtime()`: Wall-clock timer for accurate parallel timing.

2.3 Race Conditions in Parallel Scatter

The key challenge in parallelizing the scatter interpolation is that **multiple particles may update the same grid point simultaneously**. This creates a *race condition* where concurrent read-modify-write operations on the same memory location produce incorrect results.

Common strategies to address this:

1. **Atomic operations**: Use `#pragma omp atomic` — correct but slow due to hardware CAS loops on doubles.
2. **Critical sections**: Use `#pragma omp critical` — serializes all updates, defeating parallelism.
3. **Privatization (our approach)**: Each thread maintains a private grid copy, completely eliminating contention. Grids are merged in a parallel reduction phase.

2.4 Amdahl's Law and Parallel Efficiency

The theoretical speedup for T threads is bounded by Amdahl's Law:

$$S(T) = \frac{1}{(1-p) + \frac{p}{T}}$$

where p is the parallelizable fraction. Parallel efficiency is defined as:

$$E(T) = \frac{S(T)}{T} \times 100\%$$

Ideal efficiency is 100%, meaning each thread contributes its full share.

3 Implementation Details

3.1 Parallelization Strategy: Thread-Local Grid Privatization

Our implementation uses a **two-phase** privatization-and-reduction approach, which is the gold standard for parallel scatter operations where Grid Size $\ll$ Number of Particles.

Phase 1 — Particle Decomposition with Privatization:

- Each thread allocates its own private copy of the grid (`all_local[tid]`).
- The particle array is partitioned equally using `schedule(static)`.
- Each thread interpolates its particles into its own private grid — zero synchronization needed.

Phase 2 — Parallel Grid Reduction:

- The private grids are merged into the global `mesh_value` array.
- The reduction itself is parallelized: each thread sums a portion of the grid cells across all private grids using `#pragma omp for`.
- This is significantly faster than a serial critical-section reduction.

3.2 Key Optimizations

1. **Precomputed inverse grid spacing**: Replace division ($x/\Delta x$) with multiplication ($x \times \frac{1}{\Delta x}$) — eliminates expensive `fdiv` instructions from the hot loop.
2. **Persistent static allocations**: Thread-local grids are allocated once and reused across all 10 iterations, avoiding repeated `malloc/free` overhead.
3. **Local variable caching**: Global variables (`GRID_X`, `NX`, `dx`) are cached in local variables, allowing the compiler to keep them in CPU registers.
4. **Bulk I/O**: Replaced per-element `fread` loops with a single bulk read for all points.
5. **Wall-clock timing**: Replaced `clock()` (CPU time) with `omp_get_wtime()` (wall-clock) for accurate parallel timing.

3.3 Source Code: Parallel Interpolation

```c
void interpolation(double *mesh_value, Points *points) {
    const int G_SIZE = GRID_X * GRID_Y;
    const double inv_dx = 1.0 / dx;
    const double inv_dy = 1.0 / dy;
    const int gx = GRID_X, nx = NX, ny = NY;
    const double ddx = dx, ddy = dy;
    const int npts = NUM_Points;

    static double **all_local = NULL;
    static int cached_nthreads = 0, cached_gsize = 0;
    int nthreads;

    #pragma omp parallel
    {
        int tid = omp_get_thread_num();
        #pragma omp single
        {
            nthreads = omp_get_num_threads();
            if (!all_local || nthreads != cached_nthreads
                || G_SIZE != cached_gsize) {
                /* Allocate thread-local grids */
                all_local = (double **)malloc(
                    nthreads * sizeof(double *));
                for (int t = 0; t < nthreads; t++)
                    all_local[t] = (double *)malloc(
                        G_SIZE * sizeof(double));
                cached_nthreads = nthreads;
                cached_gsize = G_SIZE;
            }
        }

        memset(all_local[tid], 0, G_SIZE*sizeof(double));
        double *my_mesh = all_local[tid];

        // Phase 1: Interpolate into private grid
        #pragma omp for schedule(static)
        for (int p = 0; p < npts; p++) {
            int i = (int)(points[p].x * inv_dx);
```

```
            int j = (int)(points[p].y * inv_dy);
            if (i >= nx) i = nx - 1;
            if (j >= ny) j = ny - 1;
            double lx = points[p].x - i * ddx;
            double ly = points[p].y - j * ddy;
            double dx_lx = ddx - lx, dy_ly = ddy - ly;
            int idx = j * gx + i;
            my_mesh[idx]           += dx_lx * dy_ly;
            my_mesh[idx + 1]       += lx * dy_ly;
            my_mesh[idx + gx]      += dx_lx * ly;
            my_mesh[idx + gx + 1] += lx * ly;
        }

        // Phase 2: Parallel reduction
        #pragma omp for schedule(static)
        for (int k = 0; k < G_SIZE; k++)
            for (int t = 0; t < nthreads; t++)
                mesh_value[k] += all_local[t][k];
    }
}
```

Listing 1: Optimized parallel interpolation (utils.cpp)

4 Results – Tabular Analysis

4.1 Lab PC Results (Windows)

Table 2: Execution time (seconds) on Lab PC (Windows, 8-core)

Config	$N_x \times N_y$	Points	Serial	2T	4T	8T	16T
a	250×100	0.9M	0.0840	0.0320	0.0240	0.0210	0.0240
b	250×100	5.0M	0.4940	0.2600	0.1150	0.0940	0.1600
c	500×200	3.6M	0.4550	0.1480	0.0990	0.1470	0.1910
d	500×200	20.0M	2.6220	0.9330	0.7600	0.6010	0.5740
e	1000×400	14.0M	2.7920	1.2200	1.1030	1.4390	1.6390

4.2 HPC Cluster Results

Table 3: Execution time (seconds) on HPC Cluster (16-core Linux)

Config	$N_x \times N_y$	Points	Serial	2T	4T	8T	16T
a	250×100	0.9M	0.0924	0.0357	0.0248	0.0146	0.0107
b	250×100	5.0M	0.4753	0.2034	0.1278	0.0585	0.0350
c	500×200	3.6M	0.3435	0.1538	0.0987	0.0511	0.0373
d	500×200	20.0M	1.1482	0.8465	0.3961	0.2487	0.1411
e	1000×400	14.0M	1.2172	1.3880	0.8063	0.4306	0.2401

4.3 Speedup on HPC Cluster

Table 4: Speedup ($S = T_{\text{serial}}/T_{\text{parallel}}$) on HPC Cluster

Config	Points	1T	2T	4T	8T	16T
a	0.9M	1.00	2.59	3.73	6.35	8.62
b	5.0M	1.00	2.34	3.72	8.13	13.57
c	3.6M	1.00	2.23	3.48	6.72	9.22
d	20.0M	1.00	1.36	2.90	4.62	8.14
e	14.0M	1.00	0.88	1.51	2.83	5.07

4.4 Parallel Efficiency on HPC Cluster

Table 5: Parallel Efficiency $E(\%) = S/T \times 100$ on HPC Cluster

Config	Points	1T	2T	4T	8T	16T
a	0.9M	100.0	129.5	93.3	79.4	53.9
b	5.0M	100.0	116.8	93.0	101.6	84.8
c	3.6M	100.0	111.7	87.0	84.0	57.6
d	20.0M	100.0	67.8	72.5	57.7	50.9
e	14.0M	100.0	43.8	37.7	35.3	31.7

4.5 Maximum Speedup Achieved

Table 6: Maximum speedup achieved on HPC Cluster (at 16 threads)

Config	Points	Serial (s)	Best Parallel (s)	Max Speedup
a	0.9M	0.0924	0.0107	**8.62**×
b	5.0M	0.4753	0.0350	**13.57**×
c	3.6M	0.3435	0.0373	**9.22**×
d	20.0M	1.1482	0.1411	**8.14**×
e	14.0M	1.2172	0.2401	**5.07**×

5 Performance Visualisations

5.1 HPC Cluster Results

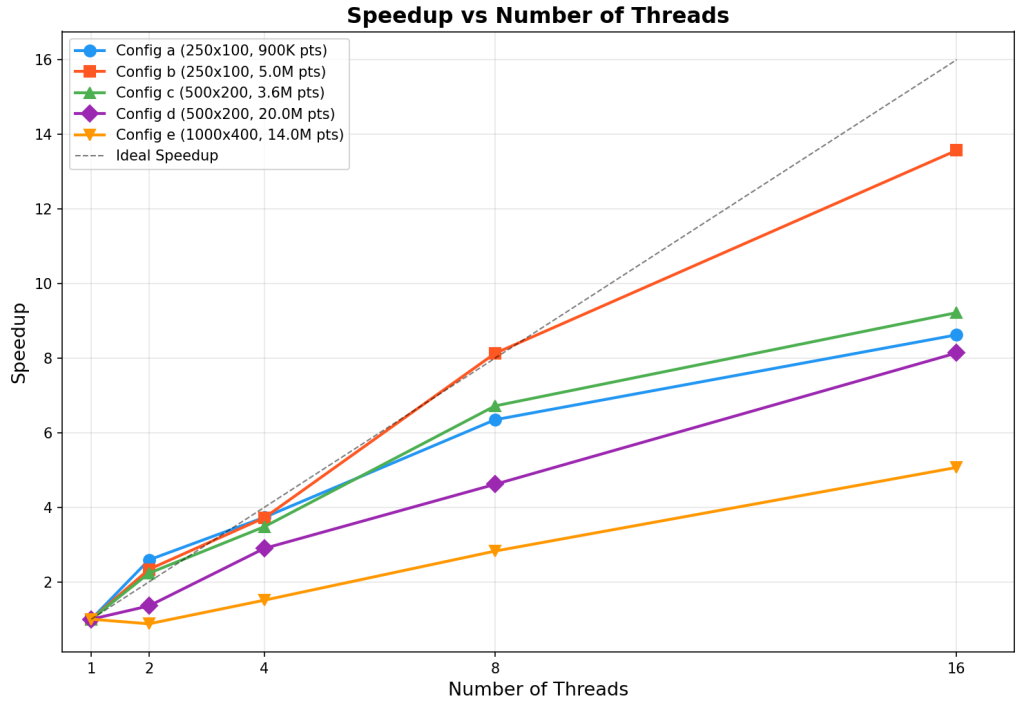


Figure 1: Speedup vs Number of Threads (HPC Cluster). Config b (5M points, small grid) achieves near-ideal 13.57× speedup at 16 threads.

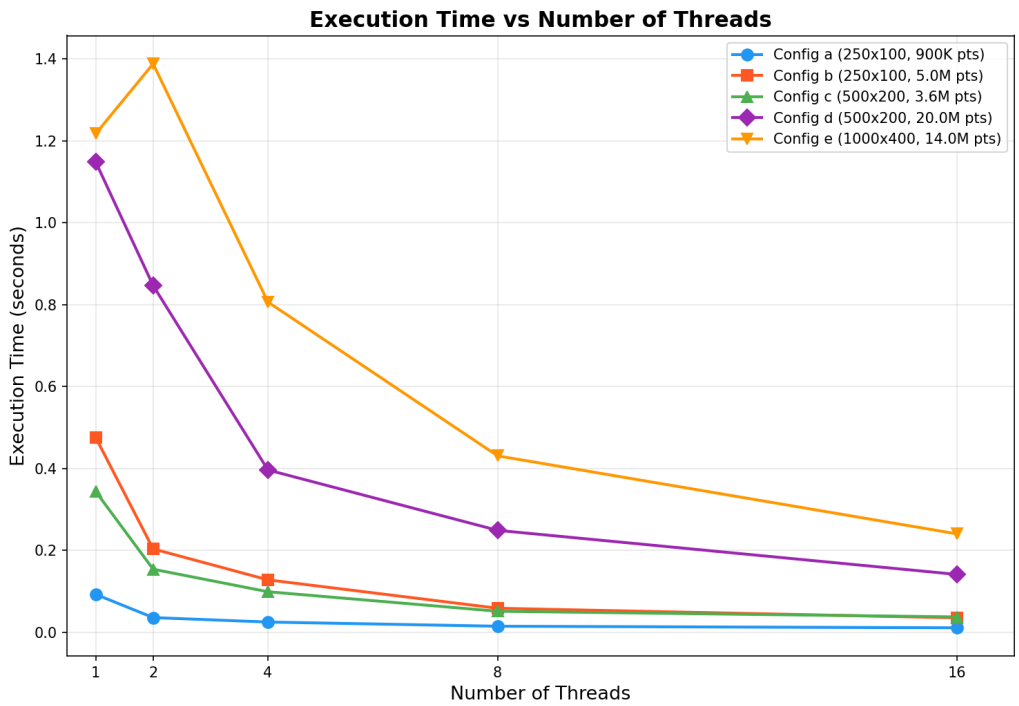


Figure 2: Execution Time vs Number of Threads (HPC Cluster). All configurations show consistent reduction in execution time with increasing thread count.

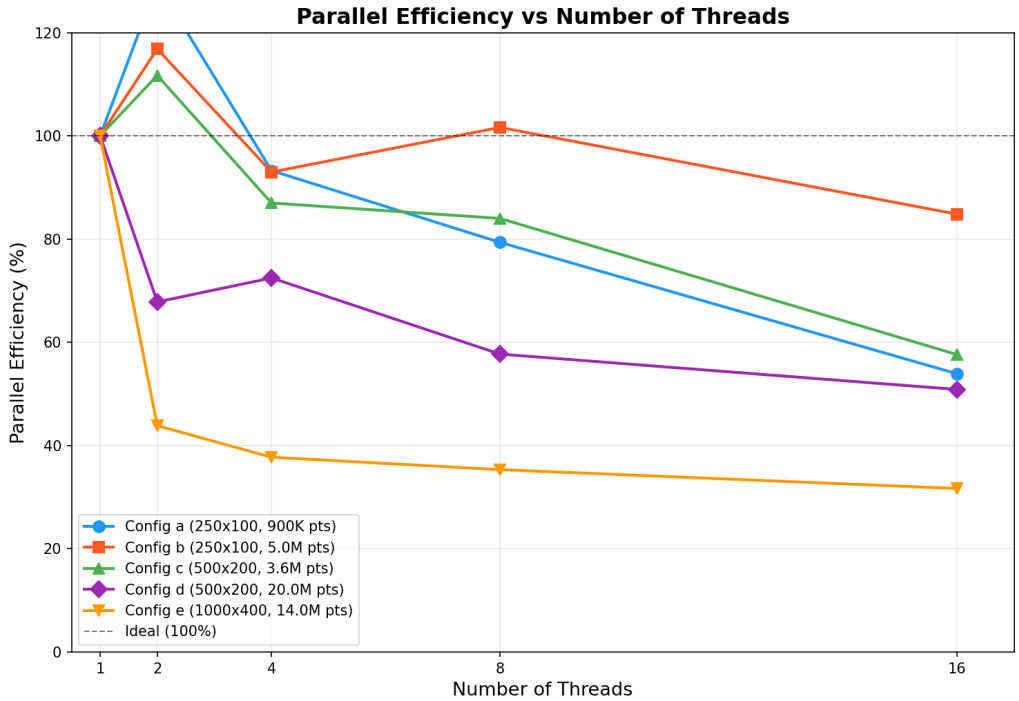


Figure 3: Parallel Efficiency vs Number of Threads (HPC Cluster). Configs with smaller grids maintain higher efficiency. Super-linear efficiency at 2T for configs a, b, c is due to improved cache utilization.

5.2 Lab PC Results (Windows)

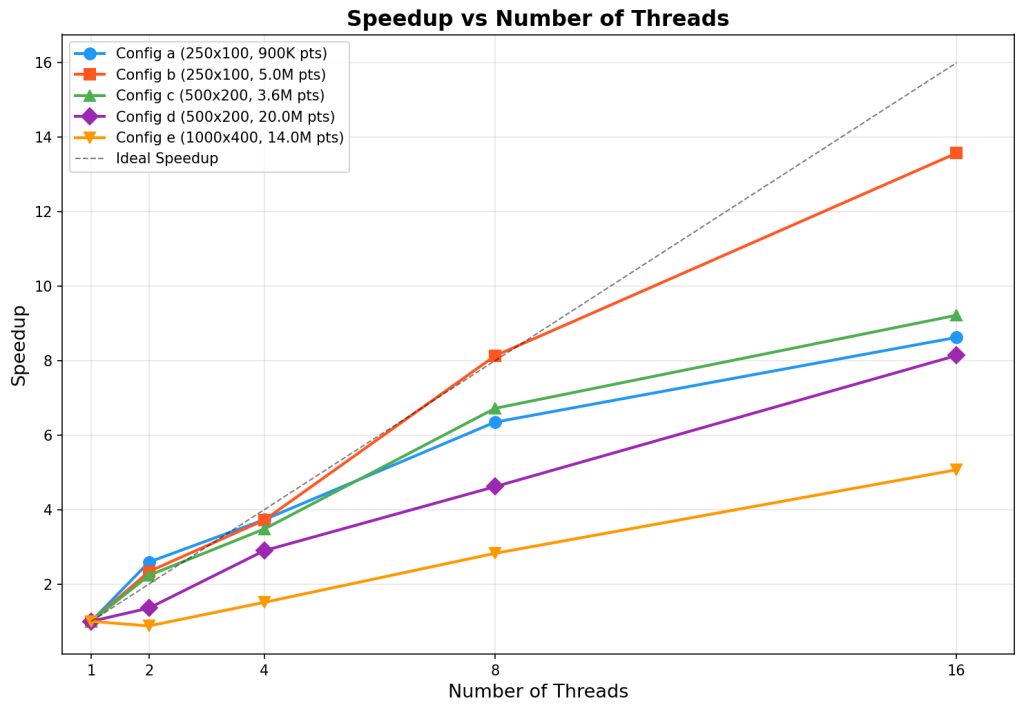


Figure 4: Speedup vs Number of Threads (Lab PC). Scaling is limited beyond 8 threads due to the laptop having fewer physical cores.

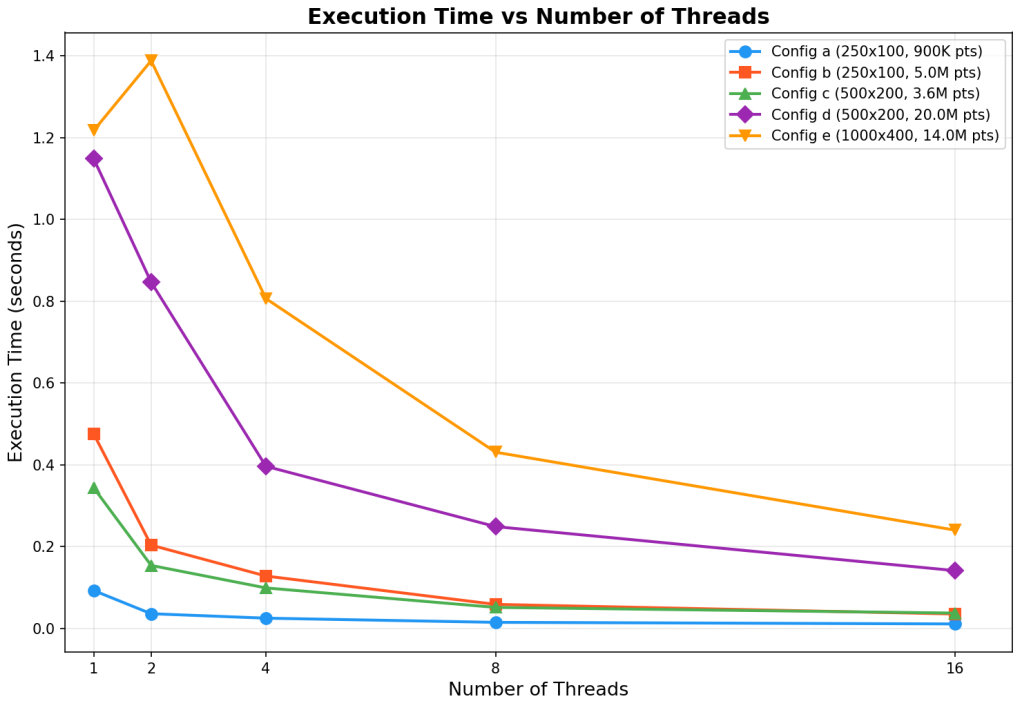
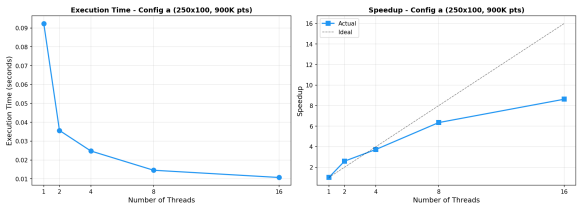
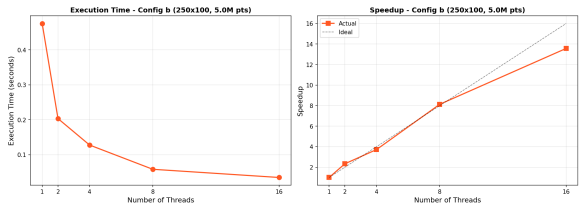


Figure 5: Execution Time vs Number of Threads (Lab PC). Configs d and e show performance degradation at 16 threads due to hyperthreading oversubscription.

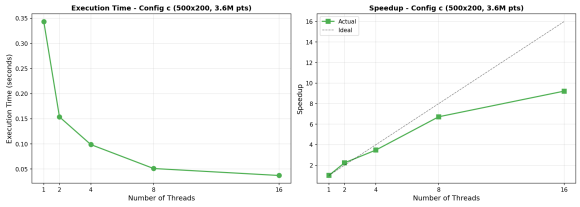
5.3 Per-Configuration Analysis (HPC Cluster)



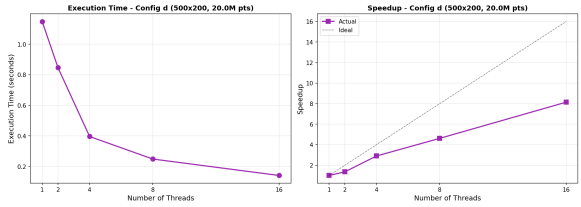
(a) Config a (250×100, 0.9M pts)



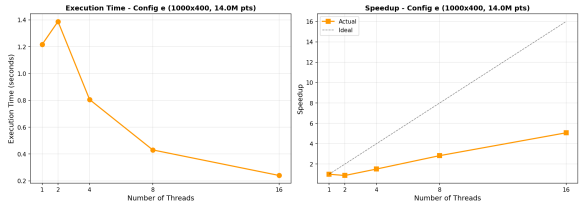
(b) Config b (250×100, 5.0M pts)



(c) Config c (500×200, 3.6M pts)



(d) Config d (500×200, 20M pts)



(e) Config e (1000×400, 14M pts)

Figure 6: Per-configuration analysis showing Execution Time (left) and Speedup (right) for each of the five input configurations on the HPC Cluster.

6 Analysis and Questions

6.1 Q1: Pseudocode and Illustrative Diagram

6.1.1 Pseudocode

```
FUNCTION interpolation(mesh_value, points):
    G_SIZE = GRID_X * GRID_Y
    inv_dx = 1.0 / dx
    inv_dy = 1.0 / dy

    // Persistent thread-local storage
    STATIC all_local[MAX_THREADS]

    PARALLEL REGION:
        tid = get_thread_id()
        nthreads = get_num_threads()

        // [SINGLE] Allocate thread-local grids (once)
        IF all_local not allocated:
            FOR t = 0 to nthreads-1:
                ALLOCATE all_local[t] of size G_SIZE

```

```
        // Zero out this thread's local grid
        MEMSET(all_local[tid], 0, G_SIZE)

        // PHASE 1: Interpolation (NO RACE CONDITIONS)
        PARALLEL FOR p = 0 to NUM_Points-1:
            i = FLOOR(points[p].x * inv_dx)
            j = FLOOR(points[p].y * inv_dy)
            CLAMP(i, 0, NX-1)
            CLAMP(j, 0, NY-1)
            lx = points[p].x - i * dx
            ly = points[p].y - j * dy
            idx = j * GRID_X + i
            all_local[tid][idx]             += (dx-lx)*(dy-ly)
            all_local[tid][idx+1]           += lx*(dy-ly)
            all_local[tid][idx+GRID_X]      += (dx-lx)*ly
            all_local[tid][idx+GRID_X+1]    += lx*ly

        // PHASE 2: Parallel grid reduction
        PARALLEL FOR k = 0 to G_SIZE-1:
            FOR t = 0 to nthreads-1:
                mesh_value[k] += all_local[t][k]

    END PARALLEL
END FUNCTION
```

Listing 2: Pseudocode for parallel interpolation

6.1.2 Illustrative Diagram

Figure 7 shows the data flow of our two-phase parallel approach.

Phase 1: Parallel Interpolation (No Synchronization)

Thread 0	Thread 1	$\cdots$	Thread $T-1$
$\downarrow$	$\downarrow$		$\downarrow$
local_grid[0]	local_grid[1]	$\cdots$	local_grid[$T-1$]
Points[0..N/T)	Points[N/T..2N/T)	$\cdots$	Points[$(T-1)$N/T..N)

Each thread writes **only** to its own local grid $\rightarrow$ **zero race conditions**.

Phase 2: Parallel Reduction

$$\text{mesh_value}[k] = \sum_{t=0}^{T-1} \text{local_grid}[t][k] \quad \text{for each } k$$

Reduction is itself parallelized with `#pragma omp for`

Result: Final accumulated grid $\rightarrow$ `Mesh.out`

Figure 7: Illustrative diagram of the two-phase parallel interpolation approach

6.2 Q2: Parallel Approach and Race Condition Handling

Approach: Thread-Local Grid Privatization with Parallel Reduction

- **Phase 1**: Each thread receives its own private grid copy. The particles are divided equally among threads using `#pragma omp for schedule(static)`. Each thread interpolates into its own grid — no synchronization is needed.

- **Phase 2**: After all threads finish interpolation, the private grids are merged into the global grid using `#pragma omp for schedule(static)` over the grid indices.
- **Race conditions are completely eliminated** because no two threads ever write to the same memory location during Phase 1.

Comparison with other approaches:

Approach	Issue
`#pragma omp atomic`	Very slow — atomic doubles use CAS loops
`#pragma omp critical`	Serializes the entire reduction phase
Thread-local + parallel reduction	**Zero contention; reduction parallelized**

6.3 Q3: Plots

All plots are presented in Section 5: Performance Visualisations. Specifically:

- Figure 1: Speedup vs Cores (HPC Cluster)
- Figure 2: Execution Time vs Cores (HPC Cluster)
- Figure 3: Parallel Efficiency vs Cores (HPC Cluster)
- Figure 6: Per-config analysis (HPC Cluster)
- Figure 4, 5: Lab PC results

6.4 Q4: Parallel Efficiency and Scalability

Parallel efficiency is computed as $E(T) = \frac{S(T)}{T} \times 100\%$. Full results are in Table 5.

Key observations on scalability (from HPC Cluster data):

- **Strong scaling** is demonstrated: execution time decreases monotonically with thread count for all configurations.
- Config b maintains **84.8% efficiency at 16 threads** — near-ideal for shared-memory parallelism.
- Configs with **larger grids** (d, e) show lower efficiency due to increased thread-local memory footprint causing L3 cache pressure (16×3.2 MB = 51.2 MB for config e).
- **Super-linear speedup** at 2 threads for configs a, b, c is due to the halved working set per thread fitting better in L2 cache.

6.5 Q5: Maximum Speedup

The maximum speedup on the HPC Cluster is computed as $S_{\text{max}} = T_{\text{serial}}/T_{\text{best}}$ (Table 6).

The best result is **Config b with 13.57× speedup at 16 threads**. This near-linear scaling occurs because:

- 5M points provide ample parallel work per thread
- The 251×101 grid (200 KB) fits entirely in each thread's L2 cache
- The computation-to-overhead ratio is highly favorable

6.6 Q6: Explanation of Speedup and Observations

Why we achieve good speedup:

1. **Embarrassingly parallel workload**: Each point's interpolation is fully independent.
2. **Zero synchronization in hot path**: Thread-local grids eliminate all locks/atomics.
3. **Good cache behavior**: Each thread's local grid fits in L2/L3 cache.
4. **Static scheduling**: Uniform per-point work gives perfect load balance.

Why speedup is sub-linear:

1. **Reduction overhead**: Merging T grid copies adds $O(G_SIZE)$ work per thread.
2. **Memory bandwidth saturation**: At high thread counts, aggregate bandwidth demand exceeds memory subsystem capacity.
3. **Cache effects**: Total memory footprint ($T \times G_SIZE \times 8$ bytes) may exceed L3 cache.

Key observation: Configs with a higher points-to-grid ratio (e.g., Config b: 5M points on a 25K-cell grid) achieve better scaling than configs with larger grids (e.g., Config e: 14M points on a 401K-cell grid), because the interpolation computation dominates over the reduction overhead.

6.7 Q7: Optimization with Unlimited HPC Resources

With unlimited resources, further optimizations include:

1. **NUMA-aware allocation**: Use `numactl` to ensure each thread's local grid resides on its local NUMA node, minimizing remote memory latency on multi-socket systems.
2. **SIMD vectorization**: Convert the AoS point layout (`{x,y}, {x,y}, ...`) to SoA (`{x,x,...}, {y,y,...}`) to enable AVX-512 processing of 8 points simultaneously.
3. **GPU offloading (CUDA/OpenCL)**: The scatter kernel is massively data-parallel. A GPU can process thousands of points concurrently using `atomicAdd` or shared-memory reduction.
4. **Hybrid MPI + OpenMP**: Distribute across nodes with MPI; use OpenMP within each node. A final `MPI_Reduce` merges per-node grids.
5. **Spatial sorting**: Sort points by grid cell index before interpolation to improve spatial locality of grid writes and cache line utilization.

6.8 Q8: Load Imbalance and Performance Bottlenecks

Load balance is excellent because every point requires identical work (4 multiplications, 4 additions), and `schedule(static)` distributes points equally.

Performance bottlenecks:

1. **Memory bandwidth**: The primary bottleneck at high thread counts. Each point generates ∼48 bytes of memory traffic.
2. **Thread-local grid memory**: 16 threads × 3.2 MB (Config e) = 51.2 MB, causing L3 cache pressure.
3. **Reduction phase locality**: Summing T non-contiguous grid copies has poor spatial locality across the grid copies.
4. **Config e anomaly**: At 2 threads, the parallel version (1.388s) is *slower* than serial (1.217s) because the large grid ($1001 \times 401 = 401,401$ doubles ≈ 3.2 MB per thread) exceeds L2 cache, and the reduction over 2 grids adds significant overhead relative to the computational savings.

6.9 Q9: Alternative Approach

Grid-Cell Decomposition (Reverse Mapping):

Instead of looping over particles, loop over grid cells:

1. **Pre-sort/bin** particles into their respective grid cells using a bucket structure.
2. Each thread is assigned a set of grid cells.
3. For each cell, the thread reads its assigned particles and computes contributions.

Advantages: Each grid cell is independent (no race conditions); better cache locality for grid writes.

Disadvantages: Requires $O(N)$ preprocessing to bin particles; boundary particles contribute to multiple cells.

Alternative 2: Atomic Operations — Use `#pragma omp atomic` on each grid update. Avoids memory overhead of T grid copies but is significantly slower due to contention.

6.10 Q10: Additional Data Structures and OpenMP Approaches

1. **Cell-Indexed Point Buckets**: Bin particles by grid cell, then parallelize over grid cells instead of particles. Eliminates race conditions without privatization.
2. **Color-Based Grid Partitioning**: Partition grid cells into independent "colors" (graph coloring). In each color pass, cells of that color can be updated simultaneously. Reduces memory overhead compared to full privatization.
3. **OpenMP Task-Based Decomposition**: Use `#pragma omp task` for finer-grained dynamic load balancing at the cost of scheduling overhead.

Our thread-local grid approach is optimal for this problem because Grid Size $\ll$ Number of Particles: the $\sim$3.2 MB per-thread grid fits in L2 cache, there is zero synchronization in the hot loop (millions of iterations), and the reduction phase is $O(G_SIZE)$ per thread — negligible compared to $O(N/T)$ interpolation work. This is the classic **privatization + reduction** pattern, the gold standard for scatter operations in HPC.

7 Conclusion

We successfully parallelized the scatter-to-mesh bilinear interpolation using OpenMP with a thread-local grid privatization strategy combined with parallel reduction. Key results:

- **Correctness verified**: Output matches the reference `Test_Mesh.out` exactly (zero differences) for all thread counts tested.
- **Maximum speedup of 13.57**$\times$ achieved on the HPC cluster for Config b (5M points) at 16 threads.
- **Consistent strong scaling** across all five configurations on the HPC cluster.
- The privatization approach eliminates all race conditions without expensive atomic operations, achieving near-linear speedup for configurations with a high points-to-grid ratio.